

Import Libraries

```
In [ ]: # Import packages and libraries
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
```

Perform exploratory Data Analysis and determine Training Labels

Here, we will:

- create a column for landing outcomes
- Standardize the data
- Split into training data and test data
- Find best parameters for SVM, Classification Trees and Logistic Regression to determine best model type for making predictions based on current data.

```
In [ ]: # Create function to plot confusion matrix
```

```
def plot_confusion_matrix(y,y_predict):
    "this function plots the confusion matrix"
    from sklearn.metrics import confusion_matrix

    cm = confusion_matrix(y, y_predict)
    ax=plt.subplot()
    sns.heatmap(cm, annot=True, ax = ax); #annot=True to annotate cells
    ax.set_xlabel('Predicted labels')
    ax.set_ylabel('True labels')
    ax.set_title('Confusion Matrix');
    ax.xaxis.set_ticklabels(['did not land', 'land']); ax.yaxis.set_ticklabels(['did not land', 'landed'])
    plt.show()
```

```
In [ ]: # Load dataframes of features
filepath_outcome_labels = '/Users/frederickhuang/Documents/VSCode/PY/SpaceX Rocket Launches/api_data_with_outcome_labels.csv'
filepath_features = '/Users/frederickhuang/Documents/VSCode/PY/SpaceX Rocket Launches/api_features_one_hot_encoded.csv'
```

```
data = pd.read_csv(filepath_outcome_labels)
X = pd.read_csv(filepath_features)
```

```
X.head()
```

```
Out[ ]: FlightNumber  PayloadMass  Flights  GridFins  Reused  Legs  Block  ReusedCount  Orbit_ES-11  Orbit_GEO  ...  Serial_B1048  Serial_B1049  Serial_B1050  Serial_B1051  Serial_B1054  Serial_B1056  Serial_B1058  Serial_B1059

0          10      6123.547647      1.0      0.0      0.0      0.0      1.0          0.0      0.0      0.0  ...          0.0          0.0          0.0          0.0          0.0          0.0          0.0
1           2       525.000000      1.0      0.0      0.0      0.0      1.0          0.0      0.0      0.0  ...          0.0          0.0          0.0          0.0          0.0          0.0          0.0
2           3       677.000000      1.0      0.0      0.0      0.0      1.0          0.0      0.0      0.0  ...          0.0          0.0          0.0          0.0          0.0          0.0          0.0
3           4       500.000000      1.0      0.0      0.0      0.0      1.0          0.0      0.0      0.0  ...          0.0          0.0          0.0          0.0          0.0          0.0          0.0
4           5       3170.000000      1.0      0.0      0.0      0.0      1.0          0.0      0.0      0.0  ...          0.0          0.0          0.0          0.0          0.0          0.0          0.0
```

5 rows x 80 columns

```
In [ ]: # Create target variable and confirm data type
y = data['Class']
type(y)
```

```
Out[ ]: pandas.Series
```

Split Data

```
In [ ]: # Split data into training test data sets.
''' Do this before standardization to avoid data leakage.
This way any models later on won't have hints about what distributions should look like from the computed mean and standard deviations.'''

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=2)
```

```
In [ ]: # Standardize training data
scaler = StandardScaler()
```

```
# Scale and fit to training data, but only scale test to ideally prevent leakage
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
In [ ]: # Gauge how many test samples we're working with
y_test.shape
```

```
Out[ ]: (18,)
```

Logistic Regression, Data validation

```
In [ ]: # Set parameters, create a logistic regression object
# Create an GridSearch object with the logistic regression, and fit to training data.
```

```
parameters = {'C':[0.01,0.1,1], 'solver':['lbfgs']} # l1 lasso l2 ridge
lr=LogisticRegression(random_state = 42)
logreg_cv = GridSearchCV(estimator = lr, param_grid = parameters, cv = 10)
logreg_cv.fit(X_train, y_train)
```

```
Out[ ]: > GridSearchCV
> best_estimator_: LogisticRegression
> LogisticRegression
> Parameters
> C 0.1
> random_state 42
> penalty 'deprecated'
> l1_ratio 0.0
> dual False
> tol 0.0001
> fit_intercept True
> intercept_scaling 1
> class_weight None
> solver 'lbfgs'
> max_iter 100
> verbose 0
> warm_start False
> n_jobs None
> Fitted attributes
```

```
In [ ]: print("ideal set of logistic regression parameters for validation: ",logreg_cv.best_params_)
print("accuracy :",logreg_cv.best_score_)

ideal set of logistic regression parameters for validation: {'C': 0.1, 'penalty': 'l2', 'solver': 'lbfgs'}
accuracy : 0.8196428571428571
```

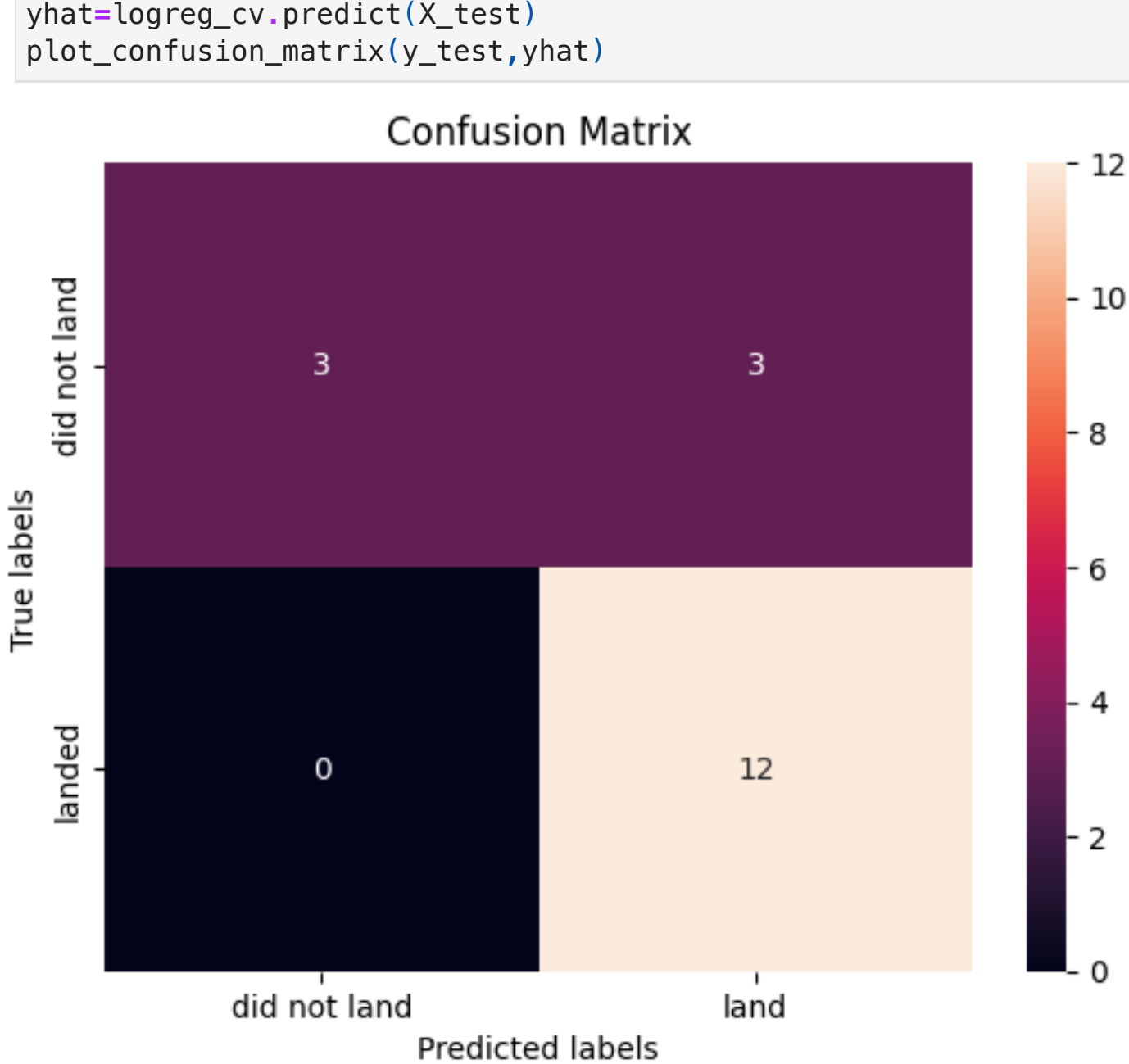
Try log reg model on test data

```
In [ ]: logreg_cv.score(X_test, y_test)
```

```
Out[ ]: 0.8333333333333334
```

Visualize Confusion Matrix

```
In [ ]: yhat=logreg_cv.predict(X_test)
plot_confusion_matrix(y_test,yhat)
```



Seem that logistic regression can distinguish between the different classes (successful rocket landing v. crash). We see that the problem is false positives.

Overview:

True Postive - 12 (True label is landed, Predicted label is also landed)

False Postive - 3 (True label is not landed, Predicted label is landed)

Support Vector Machine and parameter tuning

```
In [ ]: # Set parameters and create a support vector machine object.
parameters = {'kernel':('linear', 'rbf','poly','rbf', 'sigmoid'),
'C': np.logspace(-3, 3, 5),
'gamma':np.logspace(-3, 3, 5)}

svm = SVC()

# do some cross validation to find best parameters for this SVM, with 10 folds.
svm_cv = GridSearchCV(estimator = svm, param_grid = parameters, cv = 10)

# Fit svm_cv to training data.
svm_cv.fit(X_train, y_train)
```

```
Out[ ]: > GridSearchCV
> best_estimator_: SVC
> SVC
```

```
In [ ]: print("ideal parameters for svm training data validation ",svm_cv.best_params_)
print("accuracy :",svm_cv.best_score_)

ideal parameters for svm training data validation {'C': np.float64(1.0), 'gamma': np.float64(0.03162277660168379), 'kernel': 'sigmoid'}
accuracy : 0.8482142857142858
```

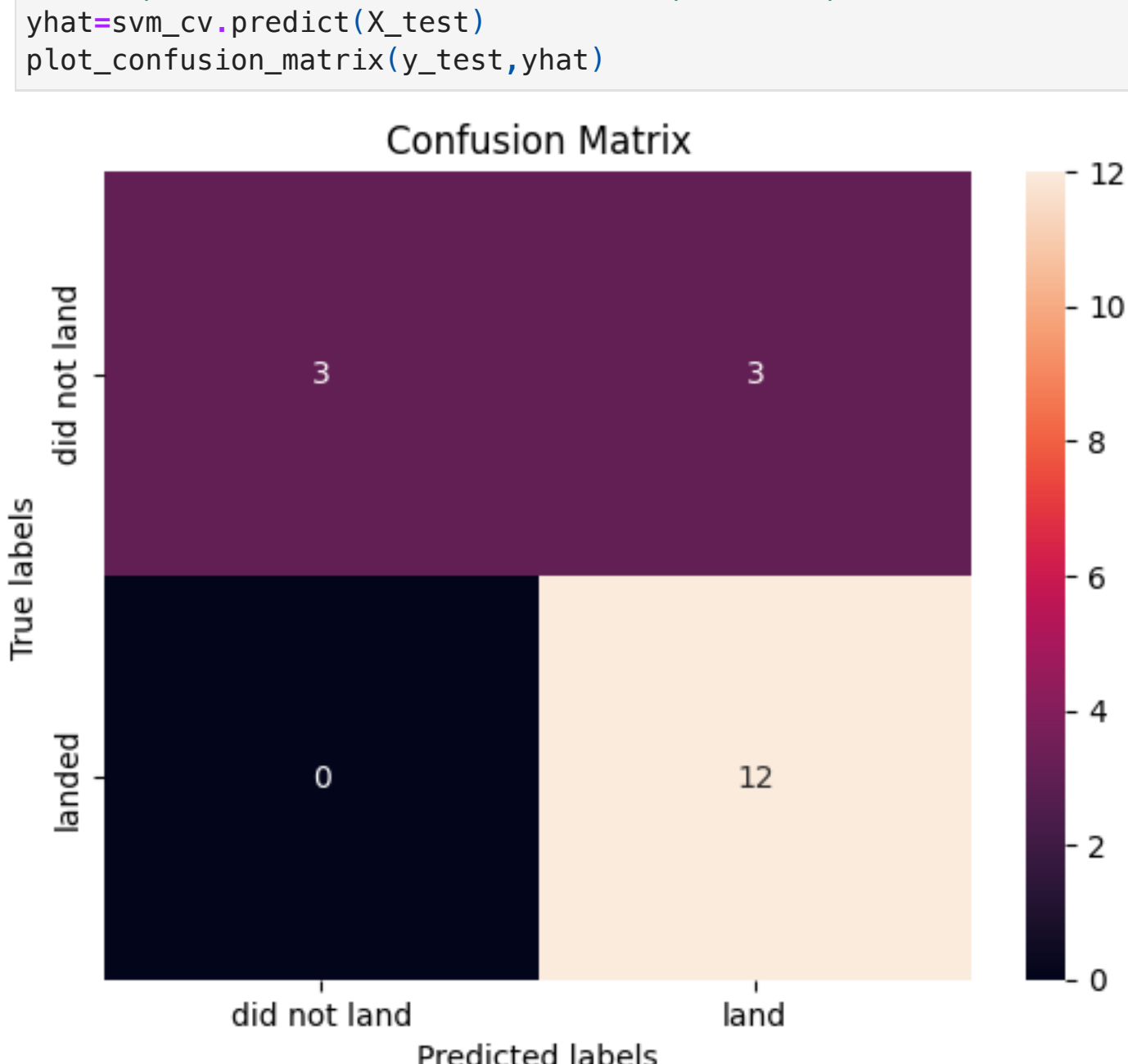
Try svm model on test data

```
In [ ]: svm_cv.score(X_test, y_test)
```

```
Out[ ]: 0.8333333333333334
```

Visualize svm model results

```
In [ ]: # Make prediction from X_test, and compare the predictions between prediction and Y_test
yhat=svm_cv.predict(X_test)
plot_confusion_matrix(y_test,yhat)
```



similar to logistic regression, SVM can distinguish between the different classes. We also see that the problem is false positives.

Overview:

True Postive - 12 (True label is landed, Predicted label is also landed)

False Postive - 3 (True label is not landed, Predicted label is landed)

Decision Tree classifier and parameter tuning

```
In [ ]: # Now instead of logistic regression or support vector machines, let's try this with a decision tree classifier now.
```

```
# Set parameters to test, and create a decision tree object.
parameters = {'criterion': ['gini', 'entropy'],
'splitter': ['best', 'random'],
'max_depth': [2m for m in range(1,10)],
'max_features': ['sqrt'],
'min_samples_leaf': [1, 2, 4],
'min_samples_split': [2, 5, 10]}
```

```
tree = DecisionTreeClassifier(random_state = 42)
```

```
# Create GridSearchCV for the decision tree with 10 fold cross validation, fit to training data
tree_cv = GridSearchCV(estimator = tree, param_grid = parameters, cv = 10)
tree_cv.fit(X_train, y_train)
```

```
Out[ ]: > GridSearchCV
> best_estimator_: DecisionTreeClassifier
> DecisionTreeClassifier
```

```
In [ ]: print("ideal parameters for decision tree model: ",tree_cv.best_params_)
print("accuracy :",tree_cv.best_score_)

ideal parameters for decision tree model: {'criterion': 'entropy', 'max_depth': 4, 'max_features': 'sqrt', 'min_samples_leaf': 1, 'min_samples_split': 2, 'splitter': 'random'}
accuracy : 0.830571428571429
```

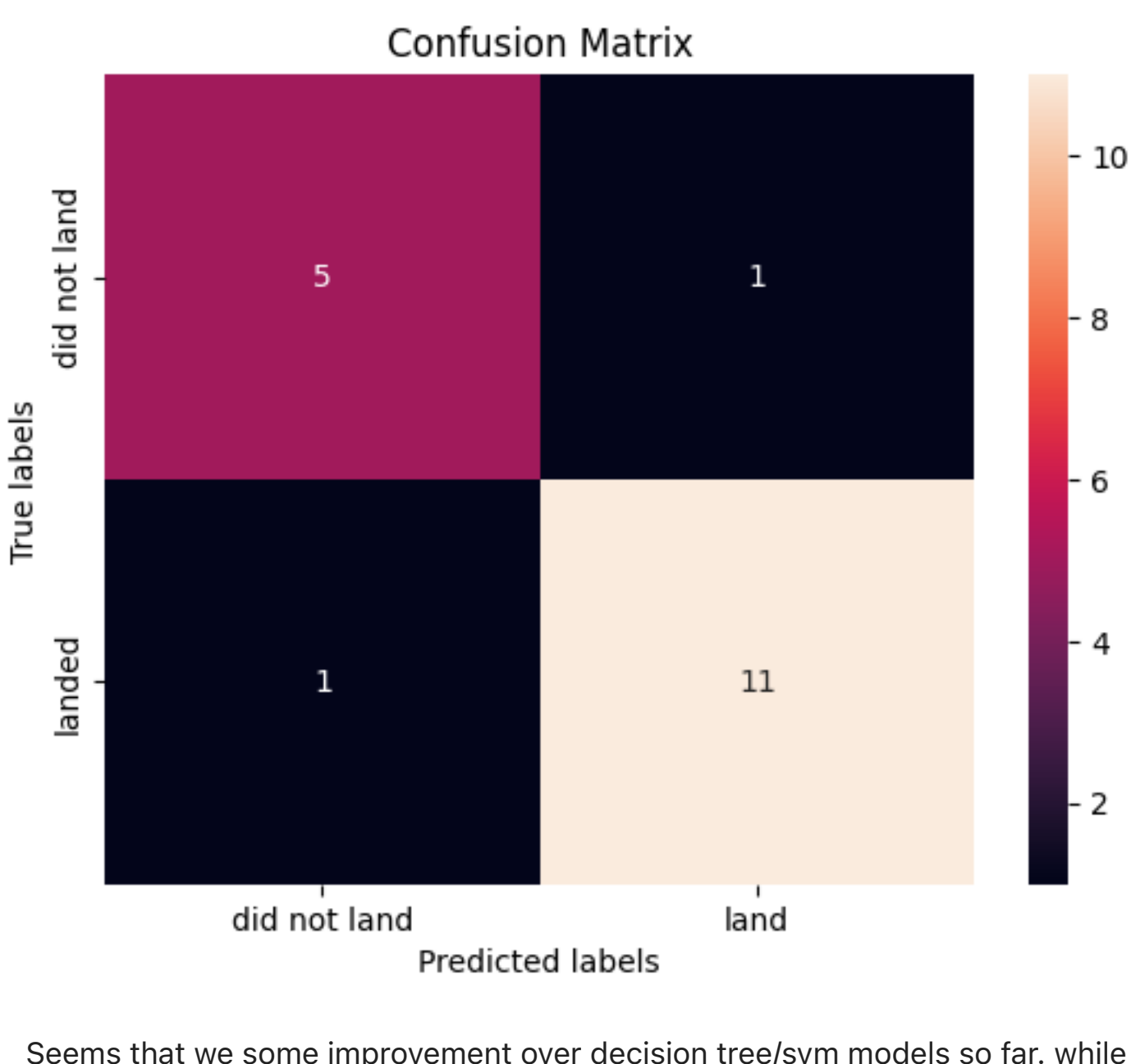
Try decision tree model on test data

```
In [ ]: tree_cv.score(X_test, y_test)
```

```
Out[ ]: 0.8888888888888888
```

Visualize results

```
In [ ]: yhat = tree_cv.predict(X_test)
plot_confusion_matrix(y_test,yhat)
```



Seems that we some improvement over decision tree/svm models so far. while distinguishing between successful landings v. rocket crashes, this comes with tradeoff of:

- introducing a false negative prediction, predicting a crash when in reality a successful landing occurred.
- however, this also did reduce instances of false positives

K-nearest neighbors

```
In [ ]: # Set parameters and createa KNN object
parameters = {'n_neighbors': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
'algorith': ['auto', 'ball_tree', 'kd_tree', 'brute'],
'p': [1,2]}
```

```
KNN = KNeighborsClassifier()
```

```
# Createa GridSearchCV object, fit to training data.
knn_cv = GridSearchCV(estimator = KNN, param_grid = parameters, cv = 10)
```

```
knn_cv.fit(X_train, y_train)
```

```
Out[ ]: > GridSearchCV
> best_estimator_: KNeighborsClassifier
> KNeighborsClassifier
```

```
In [ ]: print("ideal parameters to try for KNN: ",knn_cv.best_params_)
print("accuracy :",knn_cv.best_score_)

ideal parameters to try for KNN: {'algorithm': 'auto', 'n_neighbors': 9, 'p': 1}
accuracy : 0.8339285714285714
```

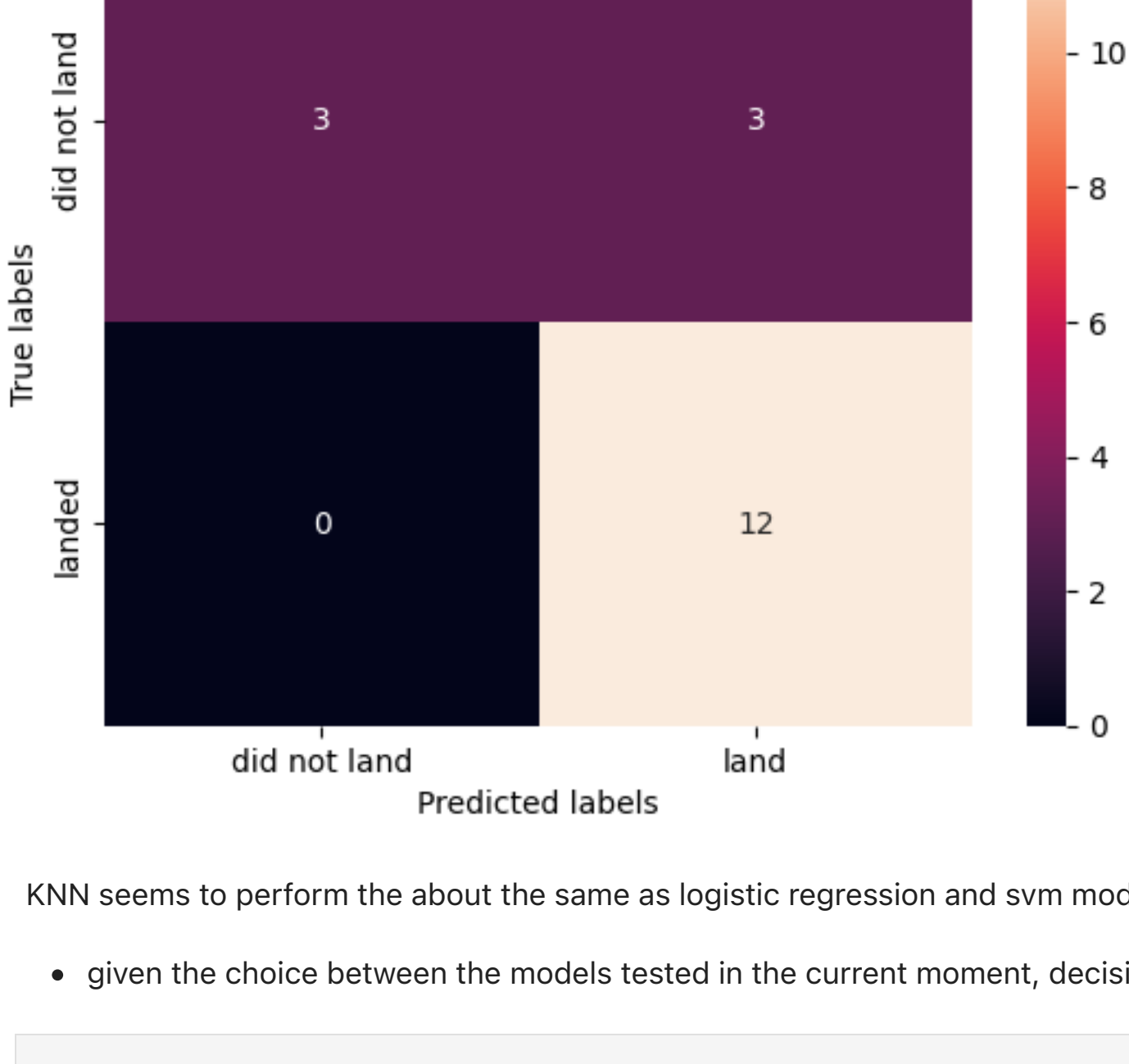
try knn on test data

```
In [ ]: knn_cv.score(X_test, y_test)
```

```
Out[ ]: 0.8333333333333334
```

Visualize Results

```
In [ ]: yhat = knn_cv.predict(X_test)
plot_confusion_matrix(y_test,yhat)
```



KNN seems to perform the about the same as logistic regression and svm models.

- given the choice between the models tested in the current moment, decision trees are probably better suited for the data we currently have.

```
In [ ]:
```